

How-To: The Plugin System

ragent can load and run third-party **plugins written for OpenAI Codex and Claude Code / Claude Desktop**. Plugin JavaScript executes on an embedded, budget-sandboxed engine (rquickjs) built into the ragent binary - **no Node.js, Deno, or Bun installation is required**. Enabled plugins contribute tools (registered as `plugin_<id>_<tool>`) and slash commands through a versioned ragent host API.

This document is the full manual: what the system is, how to install and manage plugins, how to write your own, and how to acquire plugins from the existing Claude and Codex ecosystems.

Table of Contents

- 1. Overview
- 2. Architecture
- 3. The Plugin Store
- 4. Plugin Dialects and Manifests
- 5. The ragent Host API (version 1)
- 6. Permissions and the Sandbox
- 7. Managing Plugins
- 8. Writing Your Own Plugin
- 9. Acquiring Plugins from the Claude and Codex Stores
- 10. Configuration Reference
- 11. Security Model
- 12. Error Handling and Lifecycle States
- 13. Performance Budgets
- 14. Troubleshooting
- 15. Reference

1. Overview

Why it exists

Codex and Claude Code/Desktop have growing third-party plugin ecosystems. Users who have already installed or authored plugins for those tools gain nothing from them inside ragent unless there is a compatibility layer. The plugin system provides that layer:

- it **reuses the existing ecosystem** instead of forcing authors to rewrite plugins ragent-natively;
- it lets **ragent-native plugins** be written once in JavaScript against a documented host API;
- it keeps the ragent single-binary, zero-runtime-dependency promise by running JavaScript in-process on rquickjs.

What a plugin can do

A loaded plugin contributes:

Contribution	Registered as	Visible in
Tools	<code>plugin_<id>_<tool></code>	<code>/tools</code> , the model's tool list, the normal permission system

Contribution	Registered as	Visible in
Slash commands	the declared trigger (e.g. <code>/todo-add</code>)	The <code>/</code> autocomplete menu, the command dispatcher

and can call a narrow, versioned host API to read plugin-scoped config, emit messages, write log lines, and read files **inside its own directory only**.

What a plugin cannot do

- No filesystem access outside the plugin's own directory.
- No network access, no environment variables, no process spawning.
- No `node_modules`, no npm resolution, no CommonJS `require` of arbitrary packages.
- No access to ragent credentials, memory database handles, or session internals.
- No execution at all until the plugin is explicitly **enabled** (discovery and listing parse manifests only).

Key facts

- Host API is **version 1**, additive-only.
- A plugin is **enabled when installed** (`/plugins add` records the enable flag), so it loads and contributes at the next session start; `/plugins disable` turns it off. An existing plugin whose ledger row is absent stays disabled.
- **Disable takes effect immediately** in the running session (no restart).
- Plugin code is untrusted by default; every failure is contained so it can never terminate the ragent process.

2. Architecture

The system lives in the **ragent-plugins** crate. It deliberately does **not** depend on `ragent-agent` (the tool registry) or `ragent-tui` (the command surface); the session passes closures instead, so the crate stays a bounded, testable unit.

Module	Responsibility
<code>descriptor</code> <code>manifest</code>	<code>PluginDescriptor</code> model + dialect recognition per-dialect parsing, normalisation, host-API version check
<code>store</code>	store paths, discovery scan, state ledger (<code>_state.json</code>)
<code>add</code>	<code>/plugins add</code> source handling (directory, zip/tar.gz, https URL)
<code>remove</code> <code>report</code>	<code>/plugins remove</code> store operation <code>/plugins add</code> and <code>/plugins remove</code> report rendering
<code>commands</code> <code>control</code>	<code>/plugins add remove</code> parse + dispatch glue <code>/plugins list enable disable</code>

Module	Responsibility
runtime	sandboxed JavaScript runtime (RuntimePool, SandboxContext)
host_api	versioned ragent host-API bridge
lifecycle	enable/disable/load/unload/errored lifecycle
tool_adapter	plugin tools in the session registry
command_adapter	plugin slash commands in the command surface
session	session start: discover, load, register, shutdown
harness	/plugins test isolated harness
help	usage text + subcommand metadata
error	contained error reporting

Runtime design

- `RuntimePool::checkout(budget)` returns a **fresh** `SandboxContext` pre-configured with the configured memory ceiling and an interrupt handler armed against a caller-supplied deadline. Runtimes are allocated lazily per checkout so a crashed plugin cannot taint a pooled runtime.
- Deadline interruption maps to `PluginError::Timeout`; allocation-failure aborts map to `PluginError::MemoryLimit`; uncaught JavaScript exceptions map to `PluginError::Script`. No panic path escapes.
- The context exposes **no globals beyond what `host_api` installs**: the QuickJS std/os modules are not registered and `print` is absent.

Why rquickjs

Both candidate engines (Boa and rquickjs) were evaluated against three criteria:

Criterion	rquickjs	Boa
Deadline interruption	<code>Runtime::set_interrupt_handler</code> aborts <code>while(true){}</code> within ~50 ms of the deadline	No wall-clock deadline hook
Memory ceiling	<code>Runtime::set_memory_limit(.N)</code> contains unbounded allocation loops	No allocation-budget API
JSON marshalling	JSON crosses as strings via <code>JSON.parse/JSON.stringify</code>	Equivalent

rquickjs (QuickJS) was selected because the quality of its deadline and memory-ceiling hooks is what the sandbox depends on.

3. The Plugin Store

Plugins live in a **store**. There are two store roots, scanned in ascending-priority order (project wins on plugin-id collision):

1. **user-global**: `~/.config/ragent/plugins/`

2. **project-local:** <working dir>/.`ragent/plugins/`

`plugins.store_dir` in `ragent.json`, when set, **replaces** the project leg.

Layout

Each plugin is its own subdirectory named after its plugin id:

```
.ragent/plugins/
|-- _state.json           <- per-store enable/disable state + telemetry ledger
|-- codex-weather/
|   |-- codex-plugin.json
|   `-- main.js
`-- claude-todo/
    |-- .claude-plugin/
    |   `-- plugin.json
    `-- index.js
```

The state ledger (`_state.json`)

Enable/disable state and telemetry counters persist in a per-store `_state.json` file keyed by plugin id, so state survives restarts:

```
{
  "plugins": {
    "codex-weather": {
      "enabled": true,
      "counters": {
        "loads_ok": 3,
        "load_failures": 0,
        "tool_invocations": 12,
        "tool_failures": 0,
        "consecutive_failures": 0
      }
    }
  }
}
```

Ids absent from the ledger are treated as **disabled with zero counters** (the default); `/plugins` add records a freshly installed plugin **enabled**, so it loads at the next session start until explicitly disabled.

Discovery rules

- Discovery **parses manifests only**; it never executes JavaScript.
- Unrecognised entries (plain directories, the `_state.json` ledger, dotfiles) are skipped.
- Symlinked plugin directories are followed **only when** their canonical target stays inside the store directory they were found in.
- Scanning never fails wholesale: an absent or unreadable store is treated as empty, and individual bad manifests are reported per-row instead of failing the scan.

4. Plugin Dialects and Manifests

A directory carries **exactly one** dialect. A directory whose contents match both dialects is ambiguous and rejected — **except** the multi-target case: a directory carrying *both nested* manifests (`.codex-plugin/plugin.json` **and** `.claude-plugin/plugin.json`) is a single upstream tree shipping one manifest per host, so ragent resolves it deterministically to the **Claude** dialect (the richer of the two). Any other both-match — a top-level `codex-plugin.json` beside a `claude-plugin.json`, or a `plugin.json` with a "codex" marker beside a Claude manifest — stays ambiguous.

4.1 Dialect recognition

Dialect	Recognised by
Codex	a <code>codex-plugin.json</code> , a <code>.codex-plugin/plugin.json</code> , or a <code>plugin.json</code> carrying a top-level "codex" marker field
Claude	a <code>claude-plugin.json</code> , or a <code>.claude-plugin/plugin.json</code>

4.2 Codex manifest (`codex-plugin.json`)

```
{
  "id": "codex-weather",
  "name": "Codex Weather",
  "version": "1.2.0",
  "entry": "main.js",
  "api_version": 1,
  "tools": [
    {
      "name": "get_weather",
      "description": "Return canned weather conditions for a city",
      "parameters": {
        "type": "object",
        "properties": {
          "city": { "type": "string", "description": "City name to look up" }
        },
        "required": ["city"]
      }
    }
  ],
  "commands": [
    { "name": "weather", "description": "Show weather", "usage": "/weather <city>" }
  ]
}
```

Field	Type	Required	Meaning
name	string	yes	Display name

Field	Type	Required	Meaning
version	string	yes	Version string
entry	string	yes	Entry-point JS file, relative to the plugin root
id	string	no	Explicit plugin id; derived from name when absent
api_version	u32	no	Declared host-API version; defaults to 1
tools[]	array	no	Tools declared in the manifest
commands[]	array	no	Slash commands declared in the manifest
permissions.network[]	array	no	Network grants (e.g. "outbound" becomes network.outbound)
permissions.fs	object	no	Filesystem grants - recorded as unsupported
permissions.exec	object	no	Subprocess grants - recorded as unsupported
mcp_servers	object	no	MCP transports - recorded as unsupported

4.3 Claude manifest (.claude-plugin/plugin.json)

```
{
  "name": "Claude Todo",
  "version": "0.3.0",
  "entry": "index.js",
  "api_version": 1,
  "tools": [
    {
      "name": "add_todo",
      "description": "Add an item to the plugin's todo list",
      "parameters": {
        "type": "object",
        "properties": { "text": { "type": "string" } },
        "required": ["text"]
      }
    }
  ],
  "commands": [
    { "name": "todo-add", "description": "Add a todo item", "usage": "/todo-add <text>" }
  ]
}
```

```
  ]
}
```

Field	Type	Required	Meaning
<code>name</code>	string	yes	Display name
<code>version</code>	string	no	Defaults to 0.0.0
<code>entry</code>	string	conditional	Entry point
<code>main</code>	string	conditional	Entry-point fallback
<code>server.entry</code>	string	conditional	Claude Desktop entry-point fallback
<code>id</code>	string	no	Explicit id; derived from name when absent
<code>api_version</code>	u32	no	Declared host-API version; defaults to 1
<code>tools[]</code>	array	no	Manifest-declared tools
<code>commands[]</code>	array	no	Manifest-declared commands
<code>permissions[]</code>	array	no	Requested permission strings
<code>capabilities[]</code>	array	no	Entries outside the v1 set are recorded as unsupported
<code>mcp_servers</code>	object	no	MCP transports - recorded as unsupported
<code>mounts</code>	object	no	Claude Desktop mount section - unsupported
<code>window</code>	object	no	Claude Desktop window section - unsupported

Entry resolution order for Claude: `entry`, then `main`, then `server.entry`. A manifest with none of these is rejected.

4.4 Unsupported capabilities

Manifest sections with no ragent host-API equivalent are **recorded and reported, never silently dropped**. The stable labels surfaced by `/plugins list` and `/plugins test` are:

Label	Source
<code>mcp server transports</code>	<code>mcp_servers</code> whose entry shape the bridge cannot satisfy (e.g. an array of transports); an inline object or a <code>"mcpServers"</code> : " <code>./mcp.json</code> " file reference is bridged and connected instead
<code>claude desktop mounts</code>	Claude mounts
<code>claude desktop window</code>	Claude window

Label	Source
codex permissions.fs	Codex permissions.fs
codex permissions.exec	Codex permissions.exec

4.5 Tool and command declarations

```
// Tool declaration (manifest or register_tool)
{
  "name": "get_weather",           // unique within the plugin
  "description": "...",           // shown to the model
  "parameters": { ... }           // JSON schema for the arguments object
}

// Command declaration (manifest or register_command)
{
  "name": "todo-add",              // the slash trigger, without the leading '/'
  "description": "...",           // shown in autocomplete
  "usage": "/todo-add <text>"     // optional help hint
}
```

Registered tool names are namespaced `plugin_<pluginid>_<toolname>`, so a plugin tool never collides with a built-in by construction; a *duplicate declaration within one plugin* or a *command that collides with a built-in slash trigger* is a genuine collision and is rejected.

4.6 Plugin id derivation

When id is absent, it is derived from name: split on every non-alphanumeric character, drop empties, join with -, lowercase. "Codex Weather" becomes codex-weather; an empty result becomes unnamed-plugin.

5. The reagent Host API (version 1)

Plugin code accesses exactly these capabilities through the global reagent object:

```
reagent.api_version           -> 1
reagent.plugin_id             -> string
reagent.register_tool(def)    -> { name, description, parameters, handler }
reagent.register_command(def) -> { name, description, usage, handler }
reagent.config.get(key)       -> JSON value from plugin-scoped config, or undefined
reagent.message.info(text) / .warn(text) / .error(text)
reagent.log(level, text)
reagent.plugin.read_text_file(rel_path) -> string (restricted to the plugin directory)
```

5.1 register_tool(def)

Registers a tool whose handler executes when the agent invokes it.

```

ragent.register_tool({
  name: "add_todo",
  description: "Add an item to the plugin's todo list",
  parameters: {
    type: "object",
    properties: { text: { type: "string", description: "Todo item text" } },
    required: ["text"]
  },
  handler: function (args) {
    var text = args && args.text ? String(args.text) : "";
    return { added: true, text: text };
  }
});

```

You may also declare the tool in the manifest and define a **global function named after the tool**; dispatch resolves the handler from `globalThis.__ragent_tools[name]` and falls back to a global function of the same name:

```

// The tool "get_weather" is declared in codex-plugin.json.
function get_weather(args) {
  return { city: args.city, conditions: "clear skies", temperature_c: 17 };
}

```

5.2 register_command(def)

```

ragent.register_command({
  name: "todo-add",
  description: "Add a todo item",
  usage: "/todo-add <text>",
  handler: function (text) {
    return "Added todo: " + text;
  }
});

```

Argument text typed after the trigger is passed to the handler as a plain JavaScript string; the handler's return value is surfaced as message-window text (bare strings pass through, undefined/null produce empty text, anything else is `JSON.stringify-compacted`).

5.3 Marshalling rules

Tool arguments cross into the sandbox as a JSON document bound to the scratch global `__ragent_plugin_args` and handed to the handler as a plain JavaScript value. The handler's return value is interpreted as:

Return value	Result
undefined / null	empty content
a bare string	passed through as content
{ content, isError? }	unwrapped into content plus an error flag (<code>isError: true</code> fails the call)
any other value	surfaced compactly as content and carried under tool metadata

If `JSON.stringify` of the return value throws (for example a cyclic object), the tool call fails with a serialisation error rather than panicking.

5.4 Version rule

A plugin whose declared `api_version` is **newer** than the host's is refused at enable/test time with a version-mismatch report. Equal or lower versions are accepted (v1 is additive-only, so older declarations keep working).

6. Permissions and the Sandbox

6.1 Capability groups

The host API is installed as capability groups, each keyed by one of:

`tools`, `commands`, `config`, `message`, `log`, `plugin.read_text_file`

Before installing a group, the runtime asks a **permission gate** for a decision. An ungranted group is **simply absent** from the injected ragent object. The full permission key for a capability is:

`plugin:<pluginid>:<capability>`

6.2 How gates are built

The gate is built from the plugin's declared permissions plus the `plugins.permissions` config grants. In `ragent.json`:

```
{
  "plugins": {
    "permissions": {
      "codex-weather": ["tools", "commands", "message", "log"],
      "experimental": ["*"]           // "*" grants every capability group
    }
  }
}
```

- A plugin with no matching entry gets the **full v1 surface** currently (no gate is installed). Grant an explicit list to restrict a specific plugin.
- `"*"` grants every capability group.

6.3 Sandbox budgets

Budget	Config key	Default	Meaning
Per-tool wall clock	<code>max_execution_ms</code>	5000 ms	Maximum time a single tool/command handler may run
Entry-point wall clock	<code>max_entry_ms</code>	10000 ms	Maximum time the entry point may run
Memory ceiling	<code>max_memory_mb</code>	64 MiB	Per-JavaScript-context heap ceiling

Deep recursion trips a 1 MiB engine stack before the heap ceiling, so plugin code cannot overflow the host stack via JS recursion.

6.4 What the sandbox denies

- Filesystem access outside the plugin directory (`plugin.read_text_file("../../etc/passwd")` is refused; `..` components and absolute paths are rejected before any read).
- Network access of any kind.
- Environment variables and process spawning.
- Everything not explicitly exposed through the `ragent` object.

7. Managing Plugins

The same operations are available on two surfaces that share one help text and one set of handlers:

- the **TUI slash command**: `/plugins <sub> [args...]`
- the **CLI parity surface**: `ragent plugins <sub> [args...]`

7.1 Subcommands

```
/plugins list [--verbose]           # list discovered plugins
/plugins add <source> [--force]     # install a plugin (enabled on install)
/plugins remove <pluginid>          # uninstall (refused while enabled)
/plugins enable <pluginid>          # enable, load, register tools/commands
/plugins disable <pluginid>         # unload, deregister tools/commands
/plugins test <pluginid>            # isolated harness: load + invoke each tool once
/plugins help                       # usage block
```

Form	Description
<code>list [--verbose]</code>	One row per discovered plugin: id, name, version, dialect, state (disabled/enabled/loaded/errored), tool and command counts, plus a totals summary. <code>--verbose</code> (alias <code>-v</code>) appends per-plugin telemetry counters.
<code>add <source> [--force]</code>	Install and validate a plugin; reports id and dialect. Recorded enabled , so it loads at the next session start. Refuses a duplicate id unless <code>--force</code> .
<code>remove <pluginid></code>	Uninstall from the store. Refused while the plugin is enabled.
<code>enable <pluginid></code>	Mark enabled, load into the current session, register tools and commands. States the declared permissions and the outcome.
<code>disable <pluginid></code>	Mark disabled, unload, deregister every contributed tool and command, confirming how many of each were removed. Files are not deleted.

Form	Description
test <pluginid>	Load in an isolated harness (never touching the live session), invoke each tool once with schema-derived sample arguments, and report per-step results.
help	Print the usage block. A bare /plugins or an unknown subcommand does the same, creating no files.

7.2 Sources accepted by add

- a local **directory** containing a plugin manifest;
- a local **.zip** or **.tar.gz** package file;
- an **https:// URL** pointing at a .zip/.tar.gz package (non-https URLs are refused).

A single-root archive wrapper (e.g. GitHub's <repo>--<ref>/) is descended automatically. Archives are capped at **50 MiB** and extracted with path-traversal protection (no .. or absolute entries).

7.3 Worked examples (TUI)

```
/plugins help
/plugins list
/plugins add ./vendor-plugins/codex-weather
/plugins add ./packages/claude-todo.zip
/plugins add https://example.com/plugins/plugin.tar.gz
/plugins list --verbose
/plugins enable codex-weather
/plugins test claude-todo
/plugins disable codex-weather
/plugins remove claude-todo
```

7.4 CLI parity

```
ragent plugins list
ragent plugins add ./my-plugin
ragent plugins add https://example.com/pkg.zip --force
ragent plugins enable my-plugin
ragent plugins test my-plugin
ragent plugins disable my-plugin
ragent plugins remove my-plugin
ragent plugins help
```

The CLI rewrites the TUI attribution lines: From: /plugins ... becomes a plain ragent plugins ... header and ## /plugins becomes ## ragent plugins.

7.5 Output shape

Reports are prefixed with a From: /plugins <sub> attribution line. list renders a fixed-width table with a count column per contribution kind (tools, commands, skills, agents, hooks):

ID	Name	Version	Dialect	State	Tools	C
codex-weather	Codex Weather	1.2.0	codex	loaded	1	
claude-tools	Claude Tools	2.0.0	claude	loaded	0	

followed by optional Contributions:, Unsupported capabilities:, and Errors: sections, telemetry lines in verbose mode, and a summary:

Contributions:

- claude-tools: tools []; commands []; skills [db-setup]; agents [agents/security-reviewer.md]; h

Total: 2 plugin(s) - 2 enabled, 0 disabled, 0 errored.

test renders one line per harness step:

```
[ ok ] discovery (2 ms)
[ ok ] manifest validation (0 ms)
[ ok ] version check (0 ms)
[ ok ] entry execution (5 ms)
[ ok ] sample invocation add_todo (1 ms)
```

and ends by confirming the harness was unloaded with the live session untouched.

8. Writing Your Own Plugin

8.1 Minimal Codex plugin

Create a directory with two files.

codex-plugin.json

```
{
  "id": "hello-codex",
  "name": "Hello Codex",
  "version": "1.0.0",
  "entry": "main.js",
  "api_version": 1,
  "tools": [
    {
      "name": "greet",
      "description": "Greet a person by name",
      "parameters": {
        "type": "object",
        "properties": { "who": { "type": "string" } },
        "required": ["who"]
      }
    }
  ]
}
```

main.js

```

// Entry-point top-level code runs once, when the plugin is enabled.
agent.message.info("hello-codex loaded");

// The tool "greet" is declared in the manifest; dispatch finds this global.
function greet(args) {
  var who = args && args.who ? String(args.who) : "world";
  return { greeting: "Hello, " + who + "!" };
}

```

Install and enable it:

```

/plugins add ./hello-codex
/plugins enable hello-codex

```

The model can now call `plugin_hello-codex_greet`.

8.2 Minimal Claude plugin

`.claude-plugin/plugin.json`

```

{
  "name": "Hello Claude",
  "version": "1.0.0",
  "entry": "index.js",
  "api_version": 1
}

```

`index.js`

```

agent.register_tool({
  name: "shout",
  description: "Uppercase a message",
  parameters: {
    type: "object",
    properties: { text: { type: "string" } },
    required: ["text"]
  },
  handler: function (args) {
    return String(args.text || "").toUpperCase();
  }
});

agent.register_command({
  name: "hello",
  description: "Say hello",
  usage: "/hello <name>",
  handler: function (who) {
    return "Hello, " + (who || "world") + "!";
  }
});

```

8.3 Reading plugin-scoped config

Grant the config capability, then read keys the host provides:

```
var city = ragent.config.get("default_city");    // undefined when absent
```

8.4 Emitting messages and logs

```
ragent.message.info("informational note in the message window");
ragent.message.warn("a warning");
ragent.message.error("an error");
ragent.log("debug", "goes to the host log, not the message window");
```

8.5 Reading a file inside the plugin directory

```
// Allowed: a file inside the plugin's own directory.
var data = ragent.plugin.read_text_file("data/seed.json");

// Refused: '..' components and absolute paths are rejected before any read.
// var nope = ragent.plugin.read_text_file("../../outside.txt");
```

8.6 Test before enabling

`/plugins test <id>` loads the plugin in an isolated harness, executes the entry point, and invokes each contributed tool once with schema-derived sample arguments. It never touches the live session, never writes a ledger row, and never leaves the plugin enabled. Use it as the first check after any edit.

9. Acquiring Plugins from the Claude and Codex Stores

ragent has **no built-in remote marketplace browser** - installing is always `/plugins add <source>` from a directory, a local archive, or an `https://` URL. The upstream ecosystems distribute plugins as **git repositories**, so the practical workflow is: obtain the plugin or marketplace repository, then point `/plugins add` at it.

9.1 Claude Code / Claude Desktop plugins

Claude Code plugins are bundled, shareable units that can bundle slash commands, agents, skills, hooks, and MCP servers. They are distributed as **git repositories on GitHub**, and collections of plugins from one source are published as **marketplaces** (also git repositories). Anthropic maintains an official marketplace.

Step 1 - clone the plugin or marketplace repository.

```
# A single plugin repository
git clone https://github.com/<owner>/<plugin-repo> vendor/claude-plugin

# Or the official marketplace, which contains many plugins
git clone https://github.com/anthropics/claude-plugins-official vendor/claude-plugins
```

Step 2 - inspect the tree for a Claude-dialect manifest.

A Claude-dialect plugin has either a `.claude-plugin/plugin.json` or a `claude-plugin.json` at its root:

```
ls vendor/claude-plugin/.claude-plugin/plugin.json
# or
ls vendor/claude-plugin/claude-plugin.json
```

If the repository is a marketplace, each plugin is a subdirectory; find the one you want and install that subdirectory.

Step 3 - install it.

```
/plugins add ./vendor/claude-plugin
/plugins list
/plugins enable <pluginid>
```

If the plugin declares `mcp_servers`, sections `ragent` cannot satisfy, or Desktop-only mounts/window sections, those are reported under `Unsupported capabilities`: in `ragent`'s list output - the rest of the plugin still loads.

9.2 OpenAI Codex plugins

Codex plugins are recognised by a `codex-plugin.json`, a `.codex-plugin/plugin.json` (the layout the official `openai/plugins` catalogue ships), or a `plugin.json` with a top-level "codex" marker. The Codex plugin packaging format is not yet formally standardised, so `ragent`'s recogniser treats the Codex dialect as the `name/version/entry` shape documented in section 4.2, with optional `tools[]` and `commands[]`.

Step 1 - clone the plugin repository (or download a release archive).

```
git clone https://github.com/<owner>/<codex-plugin-repo> vendor/codex-plugin
```

Step 2 - confirm the manifest.

```
ls vendor/codex-plugin/codex-plugin.json # or .codex-plugin/plugin.json
```

Step 3 - install it.

```
/plugins add ./vendor/codex-plugin
/plugins enable <pluginid>
```

9.3 Installing from a hosted package (archive or URL)

When the plugin is published as a `.zip` or `.tar.gz`, install it directly - locally or over `https`:

```
# Package a directory you already have
cd vendor && zip -r ../claude-todo.zip claude-todo && cd ..
```

```
# Host the archive yourself (any https-served location works)
# e.g. a GitHub release asset URL:
```

```
/plugins add https://github.com/<owner>/<repo>/releases/download/v1.0.0/plugin.zip
```

A GitHub source ZIP (`https://github.com/<owner>/<repo>/archive/refs/heads/main.zip`) also works: it is downloaded, extracted, the single wrapper directory (`<repo>-<ref>/`) is descended automatically, and the manifest is validated before anything lands in the store.

9.4 Adapting a plugin that is not yet ragent-compatible

Because ragent recognises the **manifest dialects**, many upstream plugins load unchanged. When one does not, the usual fixes are small:

1. **No manifest in a recognised location.** Add `codex-plugin.json`, `.codex-plugin/plugin.json`, or `.claude-plugin/plugin.json` with name, version, and entry.
2. **No JavaScript entry point.** Point entry at the plugin's JS file; a hand-written `main.js` that calls `ragent.register_tool` is enough.
3. **A capability ragent does not implement** (MCP transports, subprocess grants, Desktop mounts). Nothing to do - it is reported as unsupported and the plugin still loads for its other contributions.
4. **api_version newer than 1.** ragent's host API is version 1; reduce the declaration if the plugin only uses v1 features.

9.5 A note on trust

Enabling a third-party plugin is a **trust decision**. Before enabling:

- read the entry-point JavaScript;
- check the declared permissions and grant only what is needed via `plugins.permissions`;
- run `/plugins test <id>` first - it executes the plugin in an isolated harness without registering anything in the live session.

See section 11 for the full security posture.

9.6 Capability coverage by store

Codex and Claude plugins ship much the same broad set of contribution surfaces, but each store spells them differently and ragent bridges only some. The table below lists the known surfaces for each store provider (the Codex store `openai/plugins` and the Claude store `claude-plugins-official`, the two documents the store browser parses) and whether ragent consumes them.

“Partial” and “No” rows never block a plugin's other contributions: a surface that has an FR-025 label is recorded and reported under `Unsupported capabilities:`, and any other unrecognised section is simply ignored. The “In ragent?” column answers “if the plugin's manifest is recognised, does ragent consume this surface?”.

Capability	Codex store (openai/plugins)	Claude store (claude-plugins-official)	In ragent?
Manifest	<code>.codex-plugin/plugin.json</code>	<code>.claude-plugin/plugin.json</code>	Yes - both nested locations are recognised (plus a root <code>codex-plugin.json</code> / <code>claude-plugin.json</code> , and a <code>plugin.json</code> with a top-level "codex" marker)

Capability	Codex store (openai/plugins)	Claude store (claude-plugins-official)	Is it an agent?
JavaScript tools	- (store entries carry no entry)	entry / main / server.entry	Yes - the entry point runs on the sandbox; tools[] and register_tool register as plugin_<id>_<tool>
Slash commands	commands/	commands/, manifest commands []	Yes - prompt commands from commands/*.md, plus inline register_command commands (FR-031)
Skills	skills/ (declared skills)	skills/ (declared skills)	Yes - each declared skill directory joins skill discovery (FR-029)
MCP servers	mcpServers / .mcp.json	mcpServers / .mcp.json	Yes - bridged and connected as <plugin-id>.<server> (FR-030)
Agents / subagents	agents/	agents/	Yes - declared profiles and every agents/*.md file join agent discovery, so they appear in the agent picker and are spawnable (FR-032)

Capability	Codex store (openai/plugins)	Claude store (claude-plugins-official)	Intelligent?
Hooks	hooks.json	hooks/, hooks.json	Yes - hooks declared inline or in a hooks.json file (plugin root or hooks/) are normalised onto the session hook engine (FR-033), including the group {matcher, hooks: [...] } shape; the trigger name accepts both ragent's pre_tool_use and the Claude PreToolUse spellings; a plugin hook gets CLAUDE_PLUGIN_ROOT and the Claude event JSON on stdin, and a blocking Stop hook can feed findings back before the turn ends
App / UI surfaces	apps, .app.json, interface	-	No - Codex app/UI metadata is host-specific and ignored
Desktop integration	-	mounts, window	No - recorded as claude desktop mounts / claude desktop window
Subprocess / filesystem grants	permissions.fs, permissions.exec	-	No - recorded as codex permissions.fs / codex permissions.exec

Capability	Codex store (openai/plugins)	Claude store (claude-plugins-official)	Intelligent?
LSP servers / monitors	-	LSP servers, monitors	No - recorded as <code>claude lsp servers</code> ; a marketplace <code>*-lsp</code> stub whose repo holds no manifest (the <code>inline-lspServers</code> shape, e.g. <code>rust-analyzer-lsp</code>) installs via a manifest materialised from the marketplace document into <code>.claude-plugin/plugin.json</code> , so the “no plugin manifest found” refusal now applies only to trees no recognised manifest could be produced for
Non-JS bundle	common	common	Yes - installs and loads inertly, contributing only its bridged skills/MCP (FR-028)

Two practical consequences:

- **Codex store entries install and load.** The `openai/plugins` catalogue keeps each manifest in `.codex-plugin/plugin.json`, and `ragent` recognises that nested location just like Claude’s `.claude-plugin/plugin.json`, so `/plugins add` and `/plugins enable` both succeed. (A Codex entry whose manifest sits in some other unrecognised location still reports no plugin manifest found.)
- **Skills, commands, agents, and hooks are the surfaces that carry most value today**, because a large share of both catalogues are non-JS bundles. A plugin’s `commands/*.md` become prompt commands (registered under the bare name when free, otherwise `plugin:<id>:<name>`); its declared skills join the session’s skill discovery; its `agents/*.md` profiles join agent discovery (YAML frontmatter is accepted); and its declared hooks fire on the matching session lifecycle events.

10. Configuration Reference

The optional `plugins` block in `ragent.json`:

```
{
  "plugins": {
    "enabled": true,                // master switch; default true
  }
}
```

```

    "max_execution_ms": 5000,          // per-tool wall-clock budget
    "max_entry_ms": 10000,            // entry-point wall-clock budget
    "max_memory_mb": 64,              // per-JS-context memory ceiling
    "store_dir": null,                // optional override of the plugin store path
    "permissions": {                  // optional per-plugin permission grants
      "codex-weather": ["tools", "commands", "message", "log"]
    }
  }
}

```

Field	Type	Default	Description
enabled	bool	true	Master switch. When false, no discovery or loading happens and /plugins subcommands other than help report the subsystem is disabled.
max_execution_ms	u64	5000	Per-plugin-tool wall-clock execution budget, milliseconds.
max_entry_ms	u64	10000	Entry-point wall-clock budget, milliseconds.
max_memory_mb	u64	64	Per-JavaScript-context memory ceiling, MiB.
store_dir	string?	null	Override of the plugin store directory. When null, the store is .ragent/plugins/ (project) falling back to ~/.config/ragent/plugins/ (user-global).
permissions	map<string, string[]>	{}	Per-plugin capability grants keyed by plugin id. Use ["*"] to grant everything.

The section merges **overlay-wins**: a project-level plugins block is not discarded by an absent user-global block. With `plugins.enabled: false`, every subcommand except `help` reports the subsystem is disabled and no plugin code executes.

11. Security Model

11.1 Isolation

- Plugin code runs with **no access** to the ragent process filesystem, network stack, environment variables, or credentials other than what the host API explicitly exposes.

- The host API **never** exposes API keys, provider credentials, memory database handles, or session internals.
- The sandbox is **in-process** (no seccomp, jails, or OS-process isolation).

11.2 Installation safety

- Store scanning follows symbolic links only within the store directory itself.
- `/plugins add` from a URL downloads **over https only**, to a 50 MiB cap.
- Archives are extracted with **path-traversal protection** (no `..` or absolute entries).

11.3 Execution containment

- Discovery and `/plugins list` parse manifests only; nothing executes.
- JavaScript executes only for an **enabled** plugin during session start, or inside `/plugins test`.
- Every plugin failure is contained; a plugin can never terminate the ragent process.
- Host-level traces of capability use and gate decisions are emitted at `debug/trace` only - no plugin file contents, tool results, or message payloads are logged above that level.

11.4 Trust is explicit

Enabling a plugin is a trust decision. The enable path **states the plugin's declared permissions before completing**, and `/plugins test` lets you exercise a plugin without registering anything in the live session.

12. Error Handling and Lifecycle States

12.1 Lifecycle states

State	Meaning
disabled	Installed but inert: no code executes, no contributions (still scans and lists).
enabled	Marked enabled but not currently loaded in this session.
loaded	Enabled and loaded; contributes tools and commands.
errored	A failure prevented loading or registration; contributes nothing.

12.2 Failure handling

Failure	Effect
Manifest parse failure	Plugin marked errored with cause <code>manifest-parse</code> and the JSON error position; never loads.

Failure	Effect
Entry-point execution failure	Plugin marked errored with cause entry and the JS exception message; partial tool registration is deregistered.
Tool-invocation failure (exception, timeout, memory ceiling, bad return)	Fails the individual call and is recorded in telemetry; the plugin state is unchanged unless consecutive failures reach the auto-unload threshold (default 3), when the plugin is auto-unloaded and marked errored.
Name collision	Registration is all-or-nothing per plugin; the plugin is marked errored with cause name-collision and contributes nothing.
Version mismatch (api_version newer than host)	Refused at enable/test time with a version-mismatch report.
/plugins argument errors, unknown ids, duplicate add without --force	Reported as [err] rows in the message window; no state changes.

12.3 Telemetry counters

Recorded per plugin in the state ledger and shown by `list --verbose`:

`loads_ok`, `load_failures`, `tool_invocations`, `tool_failures`, `consecutive_failures`

13. Performance Budgets

Operation	Budget
Discovery + manifest parse for up to 50 installed plugins	under 2 s (excluding enabled plugins' entry execution)
Enabled plugin entry execution within budget	at most 200 ms amortised to session start per plugin
/plugins test on a plugin contributing up to 10 tools	under 15 s, excluding user permission prompts

Observed on a debug build: discovery + list over 50 installed plugins completes in 0.01-0.02 s; `ragent plugins test` on a 10-tool plugin completes in 0.01 s; an infinite-loop entry is contained at the configured entry budget (e.g. 0.8 s at an 800 ms budget).

14. Troubleshooting

Symptom	Cause	Fix
No plugins discovered.	Store empty or wrong location	Check <code>.ragent/plugins/</code> and <code>~/.config/ragent/plugins/</code> ; set <code>plugins.store_dir</code> if using a custom path
Plugin row shows dialect ? and version -	Manifest failed to parse	Read the <code>Errors:</code> section for the JSON position; validate the manifest
ambiguous plugin manifest	Directory matches both Codex and Claude recognition <i>and</i> is not the both-nested-manifest multi-target layout	Remove one of the two manifest forms (a plugin shipping both <code>.codex-plugin/plugin.json</code> and <code>.claude-plugin/plugin.json</code> is accepted as Claude and needs no change)
plugin requires host API vN	Declared <code>api_version</code> newer than 1	Lower the declaration to 1
Enable reports cause entry	Entry point threw or timed out	Run <code>/plugins test <id></code> for the exception/timeout detail
Enable reports cause name-collision	A command name collides with a built-in trigger, or a tool/command is declared twice by the same plugin	Rename the contribution
Tool call fails with a serialisation error	Handler returned a value <code>JSON.stringify</code> cannot serialise (e.g. cyclic)	Return a plain object or string
[err] Plugin subsystem is disabled	<code>plugins.enabled</code> is false	Set <code>plugins.enabled: true</code> in <code>ragent.json</code>
A capability is missing from the <code>ragent</code> object	The plugin's <code>plugins.permissions</code> grant omits that group	Add the capability (or <code>"*"</code>) to the grant list
mcp server transports under <code>Unsupported</code>	Plugin declares an <code>mcpServers</code> entry shape the bridge cannot satisfy (e.g. an array of transports)	Often none - an inline object or a <code>"mcpServers": { "/mcp.json" file reference is bridged and connected as <plugin-id>.<server>; only the unparseable shape is reported</code>

15. Reference

15.1 The seven subcommands

`list`, `add`, `remove`, `enable`, `disable`, `test`, `help`

15.2 Host API v1 surface (quick list)

```
ragent.api_version
ragent.plugin_id
ragent.register_tool(def)
ragent.register_command(def)
ragent.config.get(key)
ragent.message.info(text) / .warn(text) / .error(text)
ragent.log(level, text)
ragent.plugin.read_text_file(relative_path)
```

15.3 Config keys

```
plugins.enabled
plugins.max_execution_ms
plugins.max_entry_ms
plugins.max_memory_mb
plugins.store_dir
plugins.permissions.<pluginid>
```

15.4 Fixture plugins (acceptance examples)

Eight fixture plugins under `assets/plugins/fixtures/` exercise the acceptance criteria and can be used as worked examples:

Fixture	Dialect	Demonstrates
codex-weather	Codex	Manifest-declared tool + global-function handler
claude-todo	Claude	Manifest tool + command with runtime <code>register_tool/register_command</code> handlers
future-api	Codex	<code>api_version: 99</code> version-mismatch refusal
escape-attempt	Codex	Filesystem-escape refusal (<code>read_text_file("../../outside.txt")</code>)
loop-forever	Codex	Entry-budget containment of <code>while (true) {}</code>
claude-mcp	Claude	<code>mcp_servers</code> recorded as unsupported
collider	Codex	Duplicate tool-name declaration (name collision)
bad-manifest	Codex	Truncated JSON manifest-parse failure

15.5 Related commands and documents

- `/reload` - reload customisations after plugin files change.
- `/tools` - toggle tool visibility (plugin tools are always advertised while enabled).

- `ragent plugins <sub>` - CLI parity for the same subcommands.
- `docs/howtos/slashcommands/plugins.md` - the `/plugins` command reference.
- `docs/howtos/config.md` section 7.37 - the `plugins` config block.
- `specs/plugins/SPEC.md` - the full plugin-system specification.